

ui.tabs 全面详细阐述

一、核心概述

`ui.tabs` 是 NiceGUI 框架中用于实现标签页布局的核心组件套件，包含 `ui.tabs`（标签容器）、`ui.tab`（单个标签）、`ui.tab_panels`（标签内容面板容器）、`ui.tab_panel`（单个标签内容面板）四个关联元素，整体设计参考了 Quasar 的标签页 API。其核心作用是在有限的界面空间内，通过标签切换的方式组织多组相关内容，实现内容的分类展示与高效切换，支持手动点击切换、程式化切换、自定义样式与图标等丰富功能，是构建结构化、交互友好界面的关键组件。

二、核心特性与核心概念

（一）核心特性

- 组件联动机制：**`ui.tabs` 与 `ui.tab_panels` 强关联，选中某个 `ui.tab` 时，对应的 `ui.tab_panel` 会自动显示，实现标签与内容的同步切换。
- 灵活的关联方式：**支持通过 `ui.tab` 实例对象或 `name` 属性关联 `ui.tab_panel`，适配不同编程场景。
- 多形态标签支持：**单个标签（`ui.tab`）可配置独立的显示文本（`label`）、标识（`name`）和图标（`icon`），满足多样化视觉需求。
- 双向切换能力：**既支持用户手动点击标签切换，也可通过 API 程式化切换标签，适配自动化交互场景。
- 布局扩展性：**支持水平标签（默认）与垂直标签布局，可与 `ui.splitter` 等组件组合，实现复杂界面布局。
- 样式高度自定义：**通过 `classes`、`props`、`style` 等属性，可灵活调整标签容器、标签、内容面板的样式，适配不同设计体系。

（二）核心概念

- 标签容器（`ui.tabs`）：**用于承载多个 `ui.tab` 元素，管理标签的选中状态，是标签组的顶层容器，通常放置在界面顶部或侧边。
- 单个标签（`ui.tab`）：**标签的具体选项，包含标签的标识、显示文本和图标，用户点击后触发对应内容面板的显示。
- 内容面板容器（`ui.tab_panels`）：**用于承载多个 `ui.tab_panel` 元素，控制内容面板的显示 / 隐藏状态，需与 `ui.tabs` 关联使用。
- 单个内容面板（`ui.tab_panel`）：**对应某个标签的具体内容区域，每个面板需与一个 `ui.tab` 关联，未选中标签对应的面板会自动隐藏。
- 关联标识：**`ui.tab` 与 `ui.tab_panel` 的关联通过两种方式实现：
 - 直接关联：将 `ui.tab` 实例对象作为 `ui.tab_panel` 的参数；
 - 名称关联：通过 `ui.tab` 的 `name` 属性作为 `ui.tab_panel` 的关联标识。

三、核心组件详解（初始化参数、属性、方法）

（一）ui.tabs（标签容器）

1. 初始化参数

参数名	说明	类型	默认值
value	初始选中的标签，支持传入 <code>ui.tab</code> 实例、 <code>ui.tab_panel</code> 实例或标签的 <code>name</code> 属性值	Any	-
on_change	标签切换时触发的回调函数，参数为事件对象（包含当前选中标签的信息）	Callable[[ValueChangeEventArguments], Any]	-

2. 核心属性

属性名	说明	类型	
classes	组件的 CSS 类（支持 Tailwind、Quasar 类），用于调整标签容器样式	Classes[Self]	
props	组件的 Quasar 特性（HTML 属性），如设置垂直布局（ <code>vertical</code> ）	Props[Self]	
style	组件的内联 CSS 样式，用于精细调整布局	Style[Self]	
value	当前选中的标签标识（与初始化 <code>value</code> 类型一致），支持动态修改	BindableProperty	-
visible	组件是否可见（支持绑定）	BindableProperty	True
html_id	组件在 HTML DOM 中的唯一 ID（版本 2.16.0 新增）	str	-

3. 核心方法

- 绑定相关：
 - `bind_value(target_object, target_name='value', ...) -> Self`：双向绑定选中标签与目标对象属性，双方值变化时自动同步。
 - `bind_value_from(target_object, target_name='value', ...) -> Self`：单向绑定（从目标到组件），同步目标对象属性值作为选中标签。
 - `bind_value_to(target_object, target_name='value', ...) -> Self`：单向绑定（从组件到目标），同步选中标签到目标对象属性。
- 状态控制：
 - `set_value(value: Any) -> None`：编程式设置选中标签（核心方法），支持传入 `ui.tab` 实例或标签 `name`。

- `set_visibility(visible: bool) -> None`：设置组件可见性。
- 样式与布局：
 - `classes(add=None, remove=None, toggle=None, replace=None) -> Self`：动态调整 CSS 类。
 - `style(add=None, remove=None, replace=None) -> Self`：动态调整内联样式。
 - `tooltip(text: str) -> Self`：为标签容器添加提示文本。
- 其他常用：
 - `clear() -> None`：清空所有子标签（`ui.tab` 元素）。
 - `delete() -> None`：删除组件及其所有子元素。
 - `on_value_change(callback) -> Self`：添加标签切换回调（与 `on_change` 参数功能一致）。

(二) ui.tab (单个标签)

1. 初始化参数

参数名	说明	类型	默认值
name	标签的唯一标识，用于关联 <code>ui.tab_panel</code> （若未指定，默认使用 <code>label</code> 或第一个参数值）	str	-
label	标签的显示文本（若未指定，默认使用 <code>name</code> 值）	str	-
icon	标签的图标（支持 Quasar 图标库名称，如 <code>home</code> 、 <code>info</code> ）	str	-

2. 核心属性与方法

- 核心属性与 `ui.tabs` 一致（`classes`、`props`、`style`、`visible` 等），可独立调整单个标签的样式。
- 核心方法：支持 `bind_visibility`（绑定可见性）、`set_visibility`（设置可见性）等，可实现单个标签的动态显示 / 隐藏。

(三) ui.tab_panels (内容面板容器)

1. 初始化参数

参数名	说明	类型	默认值
tabs	关联的 <code>ui.tabs</code> 实例，实现标签与内容面板的联动	<code>ui.tabs</code>	-
value	初始显示的内容面板，对应 <code>ui.tabs</code> 的 <code>value</code> 参数（类型一致）	<code>Any</code>	-
on_change	内容面板切换时触发的回调函数（与 <code>ui.tabs</code> 的 <code>on_change</code> 同步）	<code>Callable[[ValueChangeEventArguments], Any]</code>	-

2. 核心属性与方法

- 核心属性与 `ui.tabs` 一致，支持 `classes`、`props`、`style` 等样式调整，通过 `props='vertical'` 可设置垂直布局。
- 核心方法：
 - `set_value(value: Any) -> None`：程式切换内容面板（与 `ui.tabs.set_value` 效果一致，会同步更新关联的 `ui.tabs` 选中状态）。
 - 绑定相关方法（`bind_value` 等）：与 `ui.tabs` 绑定后，可实现选中状态的双向同步。

(四) `ui.tab_panel`（单个内容面板）

1. 初始化参数

参数名	说明	类型	默认值
name	关联的标签标识，支持传入 <code>ui.tab</code> 实例或 <code>ui.tab</code> 的 <code>name</code> 属性值	Any	-

2. 核心属性与方法

- 核心属性：`classes`（调整面板样式，如背景色、内边距）、`style`（精细布局）、`visible`（面板独立可见性）。
- 核心方法：`clear()`（清空面板内所有子元素）、`delete()`（删除面板）等。

四、使用示例

(一) 基础用法：简单水平标签页

```
from nicegui import ui

# 创建标签容器
with ui.tabs().classes('w-full') as tabs:
    tab1 = ui.tab('标签一') # 未指定name，默认name为'标签一'
    tab2 = ui.tab('标签二')

# 创建内容面板容器，关联标签容器，初始选中tab2
with ui.tab_panels(tabs, value=tab2).classes('w-full p-4 bg-gray-50'):
    # 关联tab1的内容面板
    with ui.tab_panel(tab1):
        ui.label('标签一的内容').classes('text-lg')
        ui.button('面板内按钮', on_click=lambda: ui.notify('点击了标签一的按钮'))
    # 关联tab2的内容面板
    with ui.tab_panel(tab2):
        ui.label('标签二的内容').classes('text-lg')
        ui.input('面板内输入框', placeholder='请输入内容')

ui.run()
```

- 效果：页面顶部显示两个水平标签，默认选中“标签二”，点击标签可切换对应的内容面板，面板内支持嵌入任意 NiceGUI 组件。

(二) 进阶用法：标签带图标、名称与显示文本分离

```
from nicegui import ui

# 创建标签容器，使用name关联
with ui.tabs() as tabs:
    # name为'h'，显示文本为'首页'，图标为'home'
    ui.tab('h', label='首页', icon='home')
    # name为'a'，显示文本为'关于'，图标为'info'
    ui.tab('a', label='关于', icon='info')

# 内容面板容器，初始选中name为'h'的标签
with ui.tab_panels(tabs, value='h').classes('w-full p-4'):
    # 通过name关联标签'h'
    with ui.tab_panel('h'):
        ui.label('首页内容：欢迎访问系统').classes('text-xl text-blue-600')
    # 通过name关联标签'a'
    with ui.tab_panel('a'):
        ui.label('关于系统：基于NiceGUI开发的标签页示例').classes('text-xl text-green-600')

ui.run()
```

- 关键特性：标签的 `name` 与 `label` 分离（`name` 用于逻辑关联，`label` 用于界面显示），添加图标提升视觉效果。

(三) 编程式切换标签

```
from nicegui import ui

# 定义标签与内容的映射关系
content_map = {
    '标签1': '这是标签1的内容',
    '标签2': '这是标签2的内容',
    '标签3': '这是标签3的内容'
}

# 创建标签容器
with ui.tabs() as tabs:
    for tab_name in content_map:
        ui.tab(tab_name) # 循环创建标签，name与label均为tab_name

# 创建内容面板容器
with ui.tab_panels(tabs).classes('w-full p-4 bg-gray-50 mb-4') as panels:
    for tab_name, content in content_map.items():
        with ui.tab_panel(tab_name):
            ui.label(content).classes('text-lg')

# 添加按钮，实现编程式切换标签
ui.button('切换到标签1', on_click=lambda: panels.set_value('标签1'))
ui.button('切换到标签2', on_click=lambda: tabs.set_value('标签2')) # 也可通过tabs.set_value切换
```

```
ui.run()
```

- 关键特性：通过 `ui.tabs.set_value` 或 `ui.tab_panels.set_value` 均可实现标签切换，两者效果同步。

(四) 垂直标签与分割器组合布局

```
from nicegui import ui

# 结合ui.splitter实现垂直标签布局
with ui.splitter(value=20).classes('w-full h-80') as splitter:
    # 分割器左侧：垂直标签容器
    with splitter.before:
        with ui.tabs().props('vertical').classes('w-full bg-gray-50 p-2') as tabs:
            mail_tab = ui.tab('邮件', icon='mail')
            alarm_tab = ui.tab('提醒', icon='alarm')
            movie_tab = ui.tab('影视', icon='movie')
    # 分割器右侧：内容面板容器（垂直布局）
    with splitter.after:
        with ui.tab_panels(tabs, value=mail_tab) \
            .props('vertical').classes('w-full h-full p-4'):
            with ui.tab_panel(mail_tab):
                ui.label('邮件列表').classes('text-h4 mb-2')
                ui.list(['工作邮件', '私人邮件', '垃圾邮件'])
            with ui.tab_panel(alarm_tab):
                ui.label('提醒事项').classes('text-h4 mb-2')
                ui.list(['早上9点开会', '下午3点提交报告'])
            with ui.tab_panel(movie_tab):
                ui.label('影视推荐').classes('text-h4 mb-2')
                ui.list(['《测试电影1》', '《测试电影2》'])

ui.run()
```

- 关键特性：通过 `ui.tabs.props('vertical')` 设置垂直标签，与 `ui.splitter` 组合实现左侧标签、右侧内容的经典布局，适用于后台管理系统等场景。

五、适用场景

1. **内容分类展示**：如后台管理系统的“数据统计”“用户管理”“系统设置”等功能模块切换。
2. **表单分步填写**：将复杂表单拆分为“基本信息”“详情设置”“确认提交”等标签页，提升用户体验。
3. **数据视图切换**：如同一数据集的“表格视图”“图表视图”“卡片视图”切换。
4. **侧边栏导航**：结合垂直标签与分割器，实现侧边栏标签导航 + 主内容区的布局（类似 IDE 或管理系统界面）。
5. **多状态界面切换**：如“未登录”“已登录”“管理员模式”等不同状态下的界面内容切换。

六、注意事项

1. **关联一致性**：`ui.tab_panels` 必须与 `ui.tabs` 关联（通过 `tabs` 参数），否则标签切换无法同步内容面板。
2. **标识唯一性**：同一 `ui.tabs` 下的 `ui.tab` 其 `name` 属性必须唯一，否则会导致关联混乱。
3. **样式优先级**：通过 `classes` 传入的 Tailwind/Quasar 类优先级高于默认样式，可通过 `replace` 参数完全替

换默认类。

- 编程式切换同步：** `ui.tabs.set_value` 与 `ui.tab_panels.set_value` 效果完全同步，无需重复调用。
- 版本兼容性：** `html_id` 属性仅支持 NiceGUI 2.16.0 及以上版本，`toggle` 类操作支持 2.7.0 及以上版本。
- 垂直布局配置：** 垂直标签需同时为 `ui.tabs` 和 `ui.tab_panels` 设置 `props('vertical')`，确保布局一致性。

ui.tab 全面详细阐述

一、核心概述

`ui.tab` 是 NiceGUI 标签页布局套件（含 `ui.tabs`、`ui.tab`、`ui.tab_panels`、`ui.tab_panel`）中的基础组件，用于定义单个标签选项。它作为用户交互的入口，与 `ui.tab_panel` 一一关联，用户点击 `ui.tab` 时会触发对应的 `ui.tab_panel` 内容显示。`ui.tab` 支持自定义标识、显示文本、图标，可独立控制样式与可见性，是构建标签页界面的核心交互元素，需与 `ui.tabs`（标签容器）配合使用，实现多组内容的分类切换。

二、核心特性

- 标识与显示分离：** 支持通过 `name` 属性定义标签的逻辑标识（用于关联 `ui.tab_panel`），通过 `label` 属性定义界面显示文本，两者可独立配置，适配逻辑与视觉分离的开发需求。
- 图标增强：** 可通过 `icon` 属性配置 Quasar 图标库中的图标，提升标签的视觉辨识度与界面美观度。
- 独立样式控制：** 支持通过 `classes`、`props`、`style` 等属性单独调整单个标签的样式（如颜色、大小、间距），无需影响其他标签。
- 灵活关联方式：** 可通过自身实例对象或 `name` 属性与 `ui.tab_panel` 关联，适配不同编程场景（如动态创建标签、批量关联内容面板）。
- 状态动态控制：** 支持绑定可见性、动态显示 / 隐藏，可通过 API 灵活调整标签的交互状态。

三、初始化参数

参数名	说明	类型	默认值
name	标签的唯一逻辑标识，用于与 <code>ui.tab_panel</code> 关联；若未指定，默认使用 <code>label</code> 或第一个参数值	str	未指定时取 <code>label</code> 或标签文本
label	标签的界面显示文本；若未指定，默认使用 <code>name</code> 的值	str	与 <code>name</code> 一致
icon	标签的图标（支持 Quasar 图标库名称，如 <code>home</code> 、 <code>info</code> 、 <code>mail</code> 等）	str	-（无默认图标）

参数使用说明

- 若仅传入单个字符串参数（如 `ui.tab('首页')`），则 `name` 与 `label` 均为该字符串，图标为空。
- 若需单独配置标识与显示文本，可显式指定参数（如 `ui.tab(name='home', label='首页')`）。
- 图标参数需传入 Quasar 图标库支持的名称，无需额外引入图标资源，直接生效。

四、核心属性

`ui.tab` 继承自 NiceGUI 的 `ValueElement` 和 `Visibility` 基类，拥有以下核心属性，支持样式调整、状态控制与 DOM 标识：

属性名	说明	类型	备注
classes	标签的 CSS 类，支持 Tailwind、Quasar 类，用于调整样式（如颜色、间距）	Classes[Self]	例如 <code>classes('text-red-500 px-4')</code>
props	标签的 Quasar 特性（HTML 属性），用于扩展交互或样式	Props[Self]	例如 <code>props('disabled')</code> 禁用标签
style	标签的内联 CSS 样式，用于精细调整布局（如字体大小、边框）	Style[Self]	例如 <code>style('font-size: 16px;')</code>
visible	标签是否可见（支持绑定）	BindableProperty	默认为 <code>True</code> ，设为 <code>False</code> 时隐藏标签
html_id	标签在 HTML DOM 中的唯一 ID，用于 DOM 操作或样式定位	str	版本 2.16.0 新增
is_deleted	标签是否已被删除	bool	只读属性，用于判断组件状态
is_ignoring_events	标签是否忽略事件（如点击事件）	bool	只读属性，默认 <code>False</code>
parent_slot	标签所属的父插槽	Slot None	可设置，用于调整组件嵌套关系

五、核心方法

`ui.tab` 提供丰富的方法用于绑定数据、控制状态、调整样式，以下为常用核心方法：

（一）绑定相关方法

用于将标签的状态（如可见性）与其他对象或元素关联，实现数据同步：

1. `bind_visibility(target_object, target_name='visible', forward=None, backward=None, value=None, strict=None) -> Self`
 - 双向绑定：将标签的可见性与目标对象的指定属性关联，双方值变化时自动同步。
 - 示例：`tab.bind_visibility(store, 'show_tab')`，当 `store.show_tab` 为 `True` 时标签显示。
 - 参数 `value` 可选：若指定，仅当目标属性值等于 `value` 时标签可见（如 `value=True`）。
2. `bind_visibility_from(target_object, target_name='visible', backward=None, value=None, strict=None) -> Self`
 - 单向绑定（从目标到标签）：仅同步目标对象属性的值到标签的可见性，标签状态变化不影响目标。

3. `bind_visibility_to(target_object, target_name='visible', forward=None, strict=None) -> self`

- 单向绑定（从标签到目标）：仅同步标签的可见性到目标对象属性，目标变化不影响标签。

（二）状态控制方法

用于动态调整标签的显示状态、启用 / 禁用等：

1. `set_visibility(visible: bool) -> None`

- 直接设置标签可见性：`visible=True` 显示，`visible=False` 隐藏。
- 示例：`tab.set_visibility(False)` 隐藏标签。

2. `enable() -> None`：启用标签（允许点击交互），若标签此前被禁用（通过 `props('disabled')`），调用后恢复交互。

3. `disable() -> None`：禁用标签（禁止点击交互），等价于 `tab.props('disabled')`。

4. `delete() -> None`：删除标签及其关联的所有子元素，删除后标签从 `ui.tabs` 容器中移除。

（三）样式与布局方法

用于动态调整标签的样式和布局：

1. `classes(add=None, remove=None, toggle=None, replace=None) -> self`

- 动态添加、移除、切换或替换标签的 CSS 类。
- 示例：
 - 添加类：`tab.classes(add='bg-blue-100')`（添加蓝色背景）；
 - 移除类：`tab.classes(remove='px-4')`（移除水平内边距）；
 - 切换类：`tab.classes(toggle='text-bold')`（点击时切换粗体）。

2. `style(add=None, remove=None, replace=None) -> self`

- 动态添加、移除或替换标签的内联 CSS 样式。
- 示例：`tab.style(add='border-radius: 8px; padding: 8px;')`（添加圆角和内边距）。

3. `tooltip(text: str) -> self`：为标签添加提示文本，鼠标悬浮时显示。

- 示例：`tab.tooltip('点击查看首页内容')`。

（四）其他常用方法

1. `mark(*markers: str) -> self`：为标签添加标记，用于测试查询或依赖管理，替换现有标记。

- 示例：`tab.mark('main_tab', 'nav_tab')`。

2. `move(target_container: Element | None = None, target_index: int = -1, target_slot: str | None = None) -> None`

- 移动标签到其他容器或插槽，调整标签在 `ui.tabs` 中的排序。
- 示例：`tab.move(tabs_container, target_index=0)`（将标签移到容器第一个位置）。

3. `update() -> None`：强制更新标签在客户端的渲染状态，用于动态修改属性后同步界面。

六、使用示例

(一) 基础用法：简单标签定义

```
from nicegui import ui

with ui.tabs().classes('w-full') as tabs:
    # 仅指定标签文本（name与label均为'标签一'，无图标）
    tab1 = ui.tab('标签一')
    # 指定name、label和图标
    tab2 = ui.tab(name='setting', label='设置', icon='settings')

# 关联内容面板
with ui.tab_panels(tabs, value=tab1).classes('w-full p-4'):
    with ui.tab_panel(tab1):
        ui.label('标签一的内容')
    with ui.tab_panel('setting'): # 通过name关联tab2
        ui.label('设置页面的内容')

ui.run()
```

- 效果：两个标签横向排列，第一个标签显示“标签一”，第二个标签显示“设置”+ 设置图标，点击标签切换对应内容。

(二) 进阶用法：动态控制标签可见性

```
from nicegui import ui

# 定义存储对象，用于绑定标签可见性
class Store:
    def __init__(self):
        self.show_admin_tab = False

store = Store()

with ui.tabs().classes('w-full') as tabs:
    ui.tab('首页', icon='home')
    # 管理员标签，默认隐藏，绑定store的show_admin_tab属性
    admin_tab = ui.tab('管理员', icon='shield').bind_visibility(store, 'show_admin_tab')

with ui.tab_panels(tabs).classes('w-full p-4'):
    with ui.tab_panel('首页'):
        ui.label('普通用户可见内容')
    with ui.tab_panel('管理员'):
        ui.label('仅管理员可见内容')

# 按钮控制管理员标签显示/隐藏
ui.button('切换管理员标签', on_click=lambda: setattr(store, 'show_admin_tab', not
store.show_admin_tab))

ui.run()
```

- 效果：初始状态下“管理员”标签隐藏，点击按钮可切换其显示 / 隐藏状态，同步控制对应内容面板的可见性。

(三) 样式自定义：单个标签样式调整

```
from nicegui import ui

with ui.tabs().classes('w-full') as tabs:
    # 默认样式标签
    ui.tab('默认标签')
    # 自定义样式标签：红色文本、蓝色背景、更大内边距
    custom_tab = ui.tab('自定义样式', icon='star')
    custom_tab.classes('text-red-500 bg-blue-50 px-6 py-2')
    custom_tab.style('border-radius: 4px; font-weight: bold;')

with ui.tab_panels(tabs).classes('w-full p-4'):
    with ui.tab_panel('默认标签'):
        ui.label('默认样式内容')
    with ui.tab_panel('自定义样式'):
        ui.label('自定义样式标签对应的内容')

ui.run()
```

- 效果：第二个标签显示为红色文本、蓝色背景、圆角样式，与默认标签形成视觉区分。

(四) 禁用与启用标签

```
from nicegui import ui

with ui.tabs().classes('w-full') as tabs:
    ui.tab('可点击标签', icon='check')
    # 禁用标签：禁止点击，默认灰色显示
    disabled_tab = ui.tab('禁用标签', icon='lock')
    disabled_tab.props('disabled') # 禁用标签

with ui.tab_panels(tabs).classes('w-full p-4'):
    with ui.tab_panel('可点击标签'):
        ui.label('可正常切换的内容')
    with ui.tab_panel('禁用标签'):
        ui.label('禁用标签对应的内容（无法通过点击标签显示）')

# 按钮启用禁用标签
ui.button('启用禁用标签', on_click=lambda: disabled_tab.props(remove='disabled'))

ui.run()
```

- 效果：初始状态下“禁用标签”灰色显示，无法点击；点击按钮后，标签恢复可点击状态，可切换到对应内容面板。

七、适用场景

1. **标签页导航**：作为界面主要导航选项，如后台管理系统的“用户管理”“数据统计”“系统设置”标签。

2. **分类内容切换**：用于同一区域不同分类内容的切换，如商品详情页的“商品介绍”“规格参数”“用户评价”标签。
3. **权限控制显示**：结合可见性绑定，实现基于用户权限的标签显示 / 隐藏，如仅管理员可见的“权限配置”标签。
4. **动态界面调整**：通过 `move`、`delete` 等方法，动态调整标签顺序或移除不需要的标签，适配灵活的界面需求。
5. **增强视觉交互**：通过图标、自定义样式，提升界面美观度与用户体验，如带图标的功能分类标签。

八、注意事项

1. **关联一致性**：`ui.tab` 的 `name` 属性需与对应的 `ui.tab_panel` 关联标识一致（要么传入 `ui.tab` 实例，要么传入相同的 `name` 字符串），否则无法正常切换内容。
2. **唯一标识**：同一 `ui.tabs` 容器下的所有 `ui.tab`，其 `name` 属性必须唯一，避免关联冲突。
3. **样式优先级**：`style` 内联样式优先级高于 `classes` 中的 CSS 类，若同时设置，以 `style` 为准。
4. **禁用状态**：通过 `props('disabled')` 禁用标签后，标签无法触发点击事件，但仍可通过程式切换（`ui.tabs.set_value`）显示对应内容面板。
5. **版本兼容性**：`html_id` 属性仅支持 NiceGUI 2.16.0 及以上版本，使用时需确认框架版本。
6. **图标兼容性**：`icon` 属性依赖 Quasar 图标库，需确保使用的图标名称在 Quasar 图标库中存在（可参考 Quasar 官方图标文档）。

ui.tab_panels 全面详细阐述

一、核心概述

`ui.tab_panels` 是 NiceGUI 标签页布局套件的核心容器组件，专门用于承载 `ui.tab_panel`（单个标签内容面板），与 `ui.tabs`（标签容器）强关联，构成完整的标签页交互体系。其核心作用是管理多个内容面板的显示 / 隐藏状态，当用户点击 `ui.tabs` 中的某个 `ui.tab` 时，`ui.tab_panels` 会自动显示对应的 `ui.tab_panel`，隐藏其他面板，实现标签与内容的同步联动。`ui.tab_panels` 支持初始选中标签配置、程式切换、样式自定义、垂直布局等功能，是标签页界面中承载具体内容的核心容器。

二、核心特性

1. **强关联联动**：必须与 `ui.tabs` 实例绑定，实现标签选中状态与内容面板显示状态的自动同步，无需额外编写联动逻辑。
2. **灵活的初始配置**：支持通过 `value` 参数指定初始显示的内容面板，可传入 `ui.tab` 实例、`ui.tab_panel` 实例或标签的 `name` 属性值。
3. **双向切换支持**：既响应 `ui.tabs` 的手动点击切换，也支持通过自身 `set_value` 方法程式切换内容面板，切换状态同步回关联的 `ui.tabs`。
4. **布局多样化**：默认支持水平标签对应的垂直排列内容面板，通过 `props('vertical')` 可配置垂直标签对应的水平排列内容面板，适配不同界面布局需求。
5. **样式高度自定义**：通过 `classes`、`props`、`style` 等属性，可灵活调整容器的整体样式（如背景色、内边距、边框），也可通过 `props` 为子面板统一配置样式。
6. **事件回调支持**：提供 `on_change` 回调函数，在内容面板切换时触发，便于执行额外业务逻辑（如数据加载、状态更新）。

三、初始化参数

参数名	说明	类型	默认值
tabs	关联的 <code>ui.tabs</code> 实例，是实现标签与内容联动的核心参数，必须指定	<code>ui.tabs</code>	-
value	初始显示的内容面板标识，支持 <code>ui.tab</code> 实例、 <code>ui.tab_panel</code> 实例或标签的 <code>name</code> 属性值	<code>Any</code>	- (默认选中 <code>ui.tabs</code> 的初始 <code>value</code>)
on_change	内容面板切换时触发的回调函数，参数为 <code>ValueChangeEventArguments</code> 事件对象，包含当前选中标签的信息	<code>Callable[[ValueChangeEventArguments], Any]</code>	-

参数使用说明

- `tabs` 参数为必填项，若未关联 `ui.tabs`，`ui.tab_panels` 无法响应标签切换事件。
- `value` 参数可选，若未指定，会自动继承关联 `ui.tabs` 的 `value` 值，即初始选中标签与 `ui.tabs` 保持一致。
- `on_change` 回调与 `ui.tabs` 的 `on_change` 回调同步触发，可根据需求选择在任意一方绑定逻辑。

四、核心属性

`ui.tab_panels` 继承自 NiceGUI 的 `ValueElement` 和 `Visibility` 基类，核心属性涵盖样式控制、状态管理、DOM 标识等，具体如下：

属性名	说明	类型	备注
classes	容器的 CSS 类，支持 Tailwind、Quasar 类，用于调整容器整体样式（如背景、内边距）	<code>Classes[Self]</code>	例如 <code>classes('w-full p-4 bg-gray-50')</code>
props	容器的 Quasar 特性（HTML 属性），用于扩展功能或统一配置子面板样式	<code>Props[Self]</code>	例如 <code>props('vertical before-class=bg-white')</code>
style	容器的内联 CSS 样式，用于精细调整布局（如宽度、高度、边框）	<code>Style[Self]</code>	例如 <code>style('height: 400px; border: 1px solid #eee;')</code>
value	当前显示的内容面板标识，与初始化 <code>value</code> 类型一致，支持动态修改	<code>BindableProperty</code>	只读属性，通过 <code>set_value</code> 方法修改
visible	容器是否可见（支持绑定）	<code>BindableProperty</code>	默认为 <code>True</code> ，设为 <code>False</code> 时隐藏所有内容面板

属性名	说明	类型	备注
html_id	容器在 HTML DOM 中的唯一 ID，用于 DOM 操作或样式定位	str	版本 2.16.0 新增
is_deleted	容器是否已被删除	bool	只读属性，用于判断组件状态
is_ignoring_events	容器是否忽略事件（如切换事件）	bool	只读属性，默认 <code>False</code>
parent_slot	容器所属的父插槽	Slot None	可设置，用于调整组件嵌套关系

五、核心方法

`ui.tab_panels` 提供丰富的方法用于数据绑定、状态控制、样式调整，以下为常用核心方法：

（一）绑定相关方法

用于将容器的选中状态（当前显示的面板）与其他对象或元素关联，实现数据同步：

- `bind_value(target_object, target_name='value', forward=None, backward=None, strict=None) -> Self`
 - 双向绑定：将容器的 `value`（当前选中标签标识）与目标对象的指定属性关联，双方值变化时自动同步。
 - 示例：`panels.bind_value(store, 'current_tab')`，当 `store.current_tab` 变化时，面板自动切换，反之亦然。
- `bind_value_from(target_object, target_name='value', backward=None, strict=None) -> Self`
 - 单向绑定（从目标到容器）：仅同步目标对象属性值作为容器的选中状态，容器状态变化不影响目标。
- `bind_value_to(target_object, target_name='value', forward=None, strict=None) -> Self`
 - 单向绑定（从容器到目标）：仅同步容器的选中状态到目标对象属性，目标变化不影响容器。
- `bind_visibility(target_object, target_name='visible', ...) -> Self`
 - 双向绑定容器的可见性与目标对象属性，适配动态显示 / 隐藏整个标签内容区域的场景。

（二）状态控制方法

用于动态调整容器的显示状态、切换内容面板等：

- `set_value(value: Any) -> None`
 - 编程式切换内容面板的核心方法，支持传入 `ui.tab` 实例、`ui.tab_panel` 实例或标签的 `name` 属性值。
 - 示例：`panels.set_value('home')` 或 `panels.set_value(home_tab)`，切换到对应标签的内容面板，且关联的 `ui.tabs` 会同步选中该标签。
- `set_visibility(visible: bool) -> None`
 - 控制容器整体可见性：`visible=True` 显示所有内容面板（仅当前选中的面板可见），

`visible=False` 隐藏整个容器。

3. `enable()` -> `None`：启用容器，允许响应标签切换事件（默认启用）。
4. `disable()` -> `None`：禁用容器，禁止响应标签切换事件（但仍可通过 `set_value` 编程式切换）。
5. `delete()` -> `None`：删除容器及其所有子 `ui.tab_panel` 元素，彻底移除从界面。
6. `clear()` -> `None`：清空容器内所有 `ui.tab_panel` 元素，保留容器本身。

(三) 样式与布局方法

用于动态调整容器的样式和布局：

1. `classes(add=None, remove=None, toggle=None, replace=None) -> Self`
 - 动态添加、移除、切换或替换容器的 CSS 类。
 - 示例：
 - 添加类：`panels.classes(add='bg-blue-50 rounded-lg')`（添加蓝色背景和圆角）；
 - 替换类：`panels.classes(replace='w-3/4 mx-auto')`（替换为居中且占 3/4 宽度的样式）。
2. `style(add=None, remove=None, replace=None) -> Self`
 - 动态添加、移除或替换容器的内联 CSS 样式。
 - 示例：`panels.style(add='box-shadow: 0 2px 8px rgba(0,0,0,0.1); padding: 20px;')`（添加阴影和内边距）。
3. `props(add=None, remove=None) -> Self`
 - 动态添加或移除 Quasar 特性，常用于调整布局或子面板样式。
 - 示例：`panels.props(add='vertical after-class=overflow-auto')`（设置垂直布局，子面板溢出时可滚动）。
4. `tooltip(text: str) -> Self`：为容器添加提示文本，鼠标悬浮时显示。
 - 示例：`panels.tooltip('点击顶部标签切换内容')`。

(四) 其他常用方法

1. `on_value_change(callback) -> Self`
 - 为容器添加值变化回调（与初始化 `on_change` 参数功能一致），面板切换时触发。
 - 示例：`panels.on_value_change(lambda e: ui.notify(f'当前选中: {e.value}'))`。
2. `update()` -> `None`：强制更新容器在客户端的渲染状态，用于动态修改属性（如样式、props）后同步界面。
3. `mark(*markers: str) -> Self`：为容器添加标记，用于测试查询或依赖管理，替换现有标记。
4. `move(target_container: Element | None = None, target_index: int = -1, target_slot: str | None = None) -> None`
 - 移动容器到其他父组件或插槽，调整容器在界面中的位置。

六、使用示例

(一) 基础用法：与 ui.tabs 关联的简单标签页

```
from nicegui import ui

# 创建标签容器
with ui.tabs().classes('w-full') as tabs:
    home_tab = ui.tab('首页', icon='home')
    about_tab = ui.tab('关于', icon='info')

# 创建内容面板容器，关联标签容器，初始选中首页
with ui.tab_panels(tabs, value=home_tab).classes('w-full p-4 bg-gray-50 min-h-[300px]') as panels:
    # 首页内容面板
    with ui.tab_panel(home_tab):
        ui.label('欢迎访问首页').classes('text-xl text-blue-600')
        ui.button('首页按钮', on_click=lambda: ui.notify('点击了首页按钮'))
    # 关于页面内容面板
    with ui.tab_panel(about_tab):
        ui.label('关于我们：基于 NiceGUI 开发').classes('text-xl text-green-600')
        ui.input('请输入反馈', placeholder='您的建议...')

ui.run()
```

- 效果：页面顶部为水平标签，下方为内容面板容器，默认显示“首页”内容，点击“关于”标签可切换到对应面板，容器自带灰色背景和内边距。

(二) 进阶用法：程式化切换与事件回调

```
from nicegui import ui

# 标签与内容映射
tab_content = {
    '数据统计': '当前访问量: 1000',
    '用户管理': '当前用户数: 200',
    '系统设置': '已开启自动备份'
}

# 创建标签容器
with ui.tabs() as tabs:
    for tab_name in tab_content:
        ui.tab(tab_name)

# 创建内容面板容器，绑定切换回调
with ui.tab_panels(tabs, on_change=lambda e: ui.notify(f'切换到「{e.value}」面板')) as panels:
    for tab_name, content in tab_content.items():
        with ui.tab_panel(tab_name):
            ui.label(content).classes('text-lg')

# 程式化切换按钮
```



```

ui.button('切换到系统设置', on_click=lambda: panels.set_value('系统设置'))
ui.button('切换到数据统计', on_click=lambda: panels.set_value('数据统计'))

ui.run()

```

- 关键特性：点击标签或按钮均可切换面板，切换时触发 `on_change` 回调弹出提示，按钮通过 `set_value` 实现编程式切换，且同步更新标签选中状态。

(三) 垂直布局：与 `ui.splitter` 组合

```

from nicegui import ui

# 结合分割器实现左侧垂直标签、右侧内容面板布局
with ui.splitter(value=25).classes('w-full h-96') as splitter:
    # 分割器左侧：垂直标签容器
    with splitter.before:
        with ui.tabs().props('vertical').classes('w-full bg-gray-50 p-2') as tabs:
            mail_tab = ui.tab('邮件', icon='mail')
            alarm_tab = ui.tab('提醒', icon='alarm')
            movie_tab = ui.tab('影视', icon='movie')
    # 分割器右侧：垂直布局的内容面板容器
    with splitter.after:
        with ui.tab_panels(tabs, value=mail_tab) \
            .props('vertical').classes('w-full h-full p-4') as panels:
            with ui.tab_panel(mail_tab):
                ui.label('邮件列表').classes('text-h4 mb-2')
                ui.list(['工作邮件（3封）', '私人邮件（1封）', '垃圾邮件（0封）'])
            with ui.tab_panel(alarm_tab):
                ui.label('提醒事项').classes('text-h4 mb-2')
                ui.list(['早上9点开会', '下午3点提交报告', '晚上6点健身'])
            with ui.tab_panel(movie_tab):
                ui.label('影视推荐').classes('text-h4 mb-2')
                ui.list(['《测试电影1》', '《测试电影2》', '《测试电影3》'])

ui.run()

```

- 关键特性：通过 `props('vertical')` 为 `ui.tabs` 和 `ui.tab_panels` 均设置垂直布局，与 `ui.splitter` 组合实现经典的左侧标签导航、右侧内容展示布局，适配管理系统等场景。

(四) 样式自定义与子面板统一配置

```

from nicegui import ui

with ui.tabs().classes('w-full') as tabs:
    ui.tab('标签1', icon='star')
    ui.tab('标签2', icon='heart')

# 自定义容器样式，通过 props 为子面板设置统一样式
with ui.tab_panels(tabs) \
    .classes('w-full min-h-[200px] rounded-xl shadow-md p-6 bg-white') \

```

```
        .props('before-class=border-b-2 border-gray-200 pb-4 after-class=text-gray-700') as
panels:
    with ui.tab_panel('标签1'):
        ui.label('标签1的内容, 子面板自动应用 border-b 和 pb-4 样式').classes('text-lg')
    with ui.tab_panel('标签2'):
        ui.label('标签2的内容, 子面板自动应用 text-gray-700 样式').classes('text-lg')

ui.run()
```

- 关键特性：容器添加圆角、阴影和白色背景，通过 `props` 为所有子面板统一设置边框、内边距和文本颜色，无需单独为每个 `ui.tab_panel` 配置样式。

七、适用场景

1. **功能模块承载**：作为后台管理系统、数据可视化平台等的核心内容容器，承载不同功能模块（如用户管理、数据统计、系统设置）的具体内容。
2. **分类内容展示**：用于同一主题下不同分类内容的切换，如商品详情页的“商品介绍”“规格参数”“用户评价”，新闻页面的“国内新闻”“国际新闻”。
3. **复杂表单分步展示**：将长表单拆分为“基本信息”“详情配置”“确认提交”等步骤，通过标签页切换，提升用户填写体验。
4. **侧边栏导航布局**：与 `ui.splitter`、垂直 `ui.tabs` 组合，实现左侧标签导航、右侧内容展示的布局，适配桌面端应用。
5. **动态内容加载**：结合 `on_change` 回调，在切换面板时动态加载数据（如请求接口、查询数据库），优化页面性能。

八、注意事项

1. **关联必填性**：`tabs` 参数必须传入有效的 `ui.tabs` 实例，否则 `ui.tab_panels` 无法响应标签切换，内容面板无法正常显示 / 隐藏。
2. **标识一致性**：`ui.tab_panel` 的关联标识（`ui.tab` 实例或 `name`）必须与 `ui.tabs` 中对应的 `ui.tab` 一致，否则会出现“标签与面板不匹配”的问题。
3. **布局同步性**：当 `ui.tabs` 设置为垂直布局（`props('vertical')`）时，`ui.tab_panels` 也需设置 `props('vertical')`，确保布局样式一致。
4. **样式优先级**：`ui.tab_panels` 的 `classes/style` 控制容器整体样式，`props` 中通过 `before-class/after-class` 配置的样式作用于子面板，子面板自身的 `classes` 优先级最高。
5. **编程式切换同步**：通过 `ui.tab_panels.set_value` 切换面板时，关联的 `ui.tabs` 会自动同步选中对应的标签，无需额外调用 `ui.tabs.set_value`。
6. **版本兼容性**：`html_id` 属性仅支持 NiceGUI 2.16.0 及以上版本，`toggle` 类操作支持 2.7.0 及以上版本，使用时需确认框架版本。
7. **性能优化**：当内容面板较多或内容复杂时，可结合 `on_change` 回调实现懒加载（仅在面板切换时加载内容），避免初始加载压力过大。

ui.tab_panel 全面详细阐述

一、核心概述

`ui.tab_panel` 是 NiceGUI 标签页布局套件中承载单个标签具体内容的基础组件，需与 `ui.tab`（标签）——关联，且必须嵌套在 `ui.tab_panels`（内容面板容器）中使用。其核心作用是存储对应标签的专属内容，当关联的 `ui.tab` 被选中时，`ui.tab_panel` 会自动显示；其他状态下则隐藏，实现内容的分类展示与切换。`ui.tab_panel` 支持嵌入任意 NiceGUI 组件（文本、按钮、表单、图表等），可独立配置样式与可见性，是标签页界面中承载具体业务内容的核心元素。

二、核心特性

- 强关联标签：**通过 `ui.tab` 实例或 `name` 属性与标签强绑定，标签切换时自动同步显示 / 隐藏状态，无需额外编写联动逻辑。
- 内容高度灵活：**支持嵌套任意 NiceGUI 组件及自定义布局，适配文本展示、表单填写、数据可视化等多种内容场景。
- 独立样式控制：**可通过 `classes`、`style`、`props` 等属性单独调整面板样式（如背景色、内边距、边框），与其他面板形成视觉区分。
- 动态状态管理：**支持绑定可见性、动态清空内容、删除面板等操作，适配灵活的界面交互需求。
- 继承容器特性：**可继承 `ui.tab_panels` 配置的统一样式（如 `before-class` / `after-class`），也可通过自身属性覆盖，兼顾统一性与个性化。

三、初始化参数

参数名	说明	类型	默认值
name	关联的标签标识，用于与 <code>ui.tab</code> 建立映射关系，支持传入 <code>ui.tab</code> 实例或 <code>ui.tab</code> 的 <code>name</code> 属性值	Any	-（必填）

参数使用说明

- `name` 为必填参数，是面板与标签关联的核心依据，必须与对应 `ui.tab` 的标识一致（要么传入同一个 `ui.tab` 实例，要么传入相同的 `name` 字符串）。
- 若传入 `ui.tab` 实例，关联逻辑更直接，无需担心 `name` 重复问题；若传入 `name` 字符串，需确保该字符串与目标 `ui.tab` 的 `name` 属性完全一致。
- 示例：`ui.tab_panel(home_tab)`（通过实例关联）与 `ui.tab_panel('home')`（通过 `name` 关联），前者更推荐用于动态创建场景。

四、核心属性

`ui.tab_panel` 继承自 NiceGUI 的 `Element` 和 `Visibility` 基类，核心属性涵盖样式控制、状态管理、DOM 标识等，具体如下：

属性名	说明	类型	备注
classes	面板的 CSS 类，支持 Tailwind、Quasar 类，用于调整面板样式（如背景、内边距）	Classes[Self]	例如 <code>classes('bg-blue-50 p-4 rounded-lg')</code>
props	面板的 Quasar 特性（HTML 属性），用于扩展功能（如禁用、溢出控制）	Props[Self]	例如 <code>props('overflow-auto disabled')</code>
style	面板的内联 CSS 样式，用于精细调整布局（如宽度、高度、边框）	Style[Self]	例如 <code>style('min-height: 200px; border: 1px solid #eee;')</code>
visible	面板是否可见（支持绑定），优先级高于 <code>ui.tab_panels</code> 的切换逻辑	BindableProperty	默认为 <code>True</code> ，设为 <code>False</code> 时即使标签选中也隐藏
html_id	面板在 HTML DOM 中的唯一 ID，用于 DOM 操作或样式定位	str	版本 2.16.0 新增
is_deleted	面板是否已被删除	bool	只读属性，用于判断组件状态
parent_slot	面板所属的父插槽（即 <code>ui.tab_panels</code> 的默认插槽）	Slot None	只读属性，不可手动修改

五、核心方法

`ui.tab_panel` 提供丰富的方法用于调整样式、控制状态、管理内容，以下为常用核心方法：

（一）绑定相关方法

用于将面板的状态（如可见性）与其他对象或元素关联，实现数据同步：

- `bind_visibility(target_object, target_name='visible', forward=None, backward=None, value=None, strict=None) -> Self`
 - 双向绑定：将面板的可见性与目标对象的指定属性关联，双方值变化时自动同步。
 - 示例：`panel.bind_visibility(store, 'show_panel')`，当 `store.show_panel` 为 `True` 时面板可见。
 - 参数 `value` 可选：若指定，仅当目标属性值等于 `value` 时面板可见（如 `value='active'`）。
- `bind_visibility_from(target_object, target_name='visible', backward=None, value=None, strict=None) -> Self`
 - 单向绑定（从目标到面板）：仅同步目标对象属性值到面板的可见性，面板状态变化不影响目标。
- `bind_visibility_to(target_object, target_name='visible', forward=None, strict=None) -> Self`
 - 单向绑定（从面板到目标）：仅同步面板的可见性到目标对象属性，目标变化不影响面板。

(二) 状态与内容控制方法

用于动态调整面板的显示状态、管理面板内内容：

1. `set_visibility(visible: bool) -> None`
 - 直接控制面板可见性：`visible=True` 显示，`visible=False` 隐藏（即使关联标签被选中也不显示）。
 - 示例：`panel.set_visibility(False)` 强制隐藏面板。
2. `clear() -> None`：清空面板内所有子元素（如文本、按钮、表单等），保留面板本身。
 - 示例：点击按钮清空面板内容 `ui.button('清空', on_click=lambda: panel.clear())`。
3. `delete() -> None`：删除面板及其所有子元素，面板从 `ui.tab_panels` 容器中彻底移除。
 - 示例：`panel.delete()` 后，关联的标签仍存在，但点击后无对应内容面板显示。
4. `enable() -> None`：启用面板（允许响应交互事件，默认启用），若面板此前被禁用（通过 `props('disabled')`），调用后恢复交互。
5. `disable() -> None`：禁用面板（禁止响应交互事件，如按钮点击、输入框输入），等价于 `panel.props('disabled')`。

(三) 样式与布局方法

用于动态调整面板的样式和布局：

1. `classes(add=None, remove=None, toggle=None, replace=None) -> Self`
 - 动态添加、移除、切换或替换面板的 CSS 类。
 - 示例：
 - 添加类：`panel.classes(add='shadow-md bg-white')`（添加阴影和白色背景）；
 - 切换类：`panel.classes(toggle='border-red-500')`（点击时切换红色边框）。
2. `style(add=None, remove=None, replace=None) -> Self`
 - 动态添加、移除或替换面板的内联 CSS 样式。
 - 示例：`panel.style(add='padding: 16px; font-size: 14px;')`（调整内边距和字体大小）。
3. `props(add=None, remove=None) -> Self`
 - 动态添加或移除 Quasar 特性，常用于控制溢出、禁用等功能。
 - 示例：`panel.props(add='overflow-y-auto')`（纵向溢出时显示滚动条）。
4. `tooltip(text: str) -> Self`：为面板添加提示文本，鼠标悬浮时显示。
 - 示例：`panel.tooltip('此处显示商品详情')`。

(四) 其他常用方法

1. `update() -> None`：强制更新面板在客户端的渲染状态，用于动态修改属性（如样式、内容）后同步界面。
 - 示例：动态添加内容后调用 `panel.update()` 确保界面实时刷新。
2. `mark(*markers: str) -> Self`：为面板添加标记，用于测试查询或依赖管理，替换现有标记。
 - 示例：`panel.mark('product_panel', 'detail_panel')`。

```
3. move(target_container: Element | None = None, target_index: int = -1, target_slot: str | None = None) -> Self
```

- 移动面板到其他 `ui.tab_panels` 容器或插槽（需确保新容器关联的 `ui.tabs` 有对应标签）。
- 示例：`panel.move(new_panels_container)` 将面板移到新的内容面板容器。

六、使用示例

（一）基础用法：与 `ui.tab` 关联的简单面板

```
from nicegui import ui

# 创建标签容器
with ui.tabs().classes('w-full') as tabs:
    tab1 = ui.tab('标签一')
    tab2 = ui.tab('标签二', icon='star')

# 创建内容面板容器
with ui.tab_panels(tabs, value=tab1).classes('w-full p-4 min-h-[250px]'):
    # 标签一对应的内容面板（通过实例关联）
    with ui.tab_panel(tab1) as panel1:
        ui.label('标签一的内容').classes('text-xl')
        ui.input('输入框示例', placeholder='请输入内容')
        ui.button('面板内按钮', on_click=lambda: ui.notify('点击了标签一的按钮'))
    # 标签二对应的内容面板（通过name关联，tab2的name为'标签二'）
    with ui.tab_panel('标签二') as panel2:
        ui.label('标签二的内容（带图标）').classes('text-xl text-green-600')
        ui.list(['列表项1', '列表项2', '列表项3'])

ui.run()
```

- 效果：两个标签对应两个内容面板，默认显示“标签一”的面板，点击“标签二”可切换，面板内包含多种交互组件。

（二）进阶用法：动态控制面板可见性与内容

```
from nicegui import ui

# 存储对象，用于绑定面板可见性
class Store:
    def __init__(self):
        self.show_panel3 = False

store = Store()

# 创建标签容器
with ui.tabs().classes('w-full') as tabs:
    ui.tab('面板1')
    ui.tab('面板2')
    ui.tab('隐藏面板（需解锁）')
```

```

# 创建内容面板容器
with ui.tab_panels(tabs).classes('w-full p-4 min-h-[200px]'):
    with ui.tab_panel('面板1'):
        ui.label('普通面板，始终可见')
    with ui.tab_panel('面板2') as panel2:
        ui.label('可清空内容的面板')
        ui.button('清空当前面板', on_click=lambda: panel2.clear())
    # 绑定可见性的面板，默认隐藏
    with ui.tab_panel('隐藏面板（需解锁）').bind_visibility(store, 'show_panel3') as panel3:
        ui.label('解锁后可见的面板内容').classes('text-red-600')

# 按钮控制隐藏面板的显示/隐藏
ui.button('解锁隐藏面板', on_click=lambda: setattr(store, 'show_panel3', not
store.show_panel3))

ui.run()

```

- 关键特性：“隐藏面板（需解锁）”默认不可见，点击按钮可切换其可见性；“面板 2”支持通过按钮清空内部所有内容。

（三）样式自定义：独立配置面板样式

```

from nicegui import ui

# 创建标签容器
with ui.tabs().classes('w-full') as tabs:
    ui.tab('默认样式')
    ui.tab('自定义样式')

# 创建内容面板容器
with ui.tab_panels(tabs).classes('w-full p-4 min-h-[200px]'):
    # 默认样式面板
    with ui.tab_panel('默认样式'):
        ui.label('默认样式的内容面板')
    # 自定义样式面板
    with ui.tab_panel('自定义样式') as custom_panel:
        ui.label('自定义背景、边框和阴影的面板').classes('text-lg')
        # 配置自定义样式
        custom_panel.classes('bg-blue-50 border border-blue-200 rounded-xl shadow-md p-6')
        custom_panel.style('color: #1e40af; font-weight: 500;')

ui.run()

```

- 效果：“自定义样式”面板显示为蓝色背景、蓝色边框、圆角和阴影，文本为深蓝色，与默认样式面板形成明显区分。

（四）禁用面板与动态删除

```
from nicegui import ui

# 创建标签容器
with ui.tabs().classes('w-full') as tabs:
    ui.tab('可交互面板')
    ui.tab('禁用面板')
    ui.tab('可删除面板')

# 创建内容面板容器
with ui.tab_panels(tabs).classes('w-full p-4 min-h-[200px]'):
    with ui.tab_panel('可交互面板'):
        ui.label('面板内按钮可点击')
        ui.button('可点击按钮', on_click=lambda: ui.notify('按钮被点击'))
    # 禁用面板：内部组件无法交互
    with ui.tab_panel('禁用面板') as disabled_panel:
        ui.label('此面板已禁用，内部组件不可交互').classes('text-gray-500')
        ui.button('不可点击按钮', on_click=lambda: ui.notify('不会触发'))
        disabled_panel.props('disabled') # 禁用面板
    # 可删除面板
    with ui.tab_panel('可删除面板') as removable_panel:
        ui.label('点击按钮可删除此面板').classes('text-orange-600')
        ui.button('删除当前面板', on_click=lambda: removable_panel.delete())

ui.run()
```

- 效果：“禁用面板”内的按钮无法点击；“可删除面板”点击按钮后，面板被彻底删除，后续点击对应标签无内容显示。

七、适用场景

1. **功能模块内容承载**：作为后台管理系统中“用户管理”“数据统计”“系统设置”等标签的内容容器，承载列表、表单、图表等组件。
2. **商品 / 内容详情展示**：用于电商平台商品详情页的“商品介绍”“规格参数”“用户评价”，或新闻页面的“正文”“相关推荐”等内容展示。
3. **分步表单容器**：将复杂表单拆分为“基本信息”“联系方式”“确认提交”等步骤，每个步骤对应一个 `ui.tab_panel`，提升用户填写体验。
4. **权限控制内容显示**：结合可见性绑定，实现基于用户权限的内容显示 / 隐藏，如仅管理员可见的“权限配置”面板。
5. **动态内容管理**：通过 `clear()`、`delete()` 等方法，动态调整面板内容或移除无用面板，适配灵活的业务需求（如临时展示的通知面板）。

八、注意事项

1. **关联一致性**：`ui.tab_panel` 的 `name` 参数必须与对应 `ui.tab` 的标识一致（实例或 `name` 字符串），否则标签切换时面板无法正常显示。
2. **容器依赖**：必须嵌套在 `ui.tab_panels` 容器中使用，直接放在其他容器（如 `ui.row`、`ui.card`）中会无法响应标签切换。

3. **可见性优先级**: `ui.tab_panel` 自身的 `visible` 属性优先级高于 `ui.tab_panels` 的切换逻辑, 若设为 `False`, 即使关联标签被选中也不会显示。
4. **样式优先级**: 面板自身的 `classes/style` 优先级高于 `ui.tab_panels` 配置的统一样式 (如 `before-class`), 可通过自身属性覆盖容器样式。
5. **禁用状态影响**: 通过 `props('disabled')` 禁用面板后, 面板内所有交互组件 (按钮、输入框等) 都会被禁用, 无法响应用户操作。
6. **版本兼容性**: `html_id` 属性仅支持 NiceGUI 2.16.0 及以上版本, 使用时需确认框架版本。
7. **动态操作同步**: 通过 `delete()` 删除面板后, 关联的 `ui.tab` 不会自动删除, 需手动删除标签以保持界面一致性。